Comprehensive Technical Documentation: Defective vs. Non-Defective Nut Surface Classification

1. Introduction and Project Objective

The fundamental objective of this project is the automation of quality control within manufacturing environments. In product-based and manufacturing companies, the inspection of objects and parts for defects is traditionally a manual, labor-intensive process. Human inspectors are prone to fatigue, which introduces a high margin of error. By classifying objects automatically into two overarching categories—Defective and Non-Defective—this project seeks to entirely eliminate the need for manual classification. This transition not only accelerates the manufacturing pipeline but also ensures a consistent and objective quality standard. The data used for this project is the "Nut Surface Defect Dataset," retrieved from Kaggle (https://www.kaggle.com/datasets/weihaoreal/nut-surface-defect-dataset).

2. Model Architecture and Selection Rationale

2.1 Why EfficientNet-B0?

When selecting a deep learning architecture for an industrial application, there is an inherent trade-off between computational complexity and predictive accuracy.

YOLO v9 (You Only Look Once, version 9) is a highly advanced, state-of-the-art model designed primarily for complex object detection, bounding box localization, and real-time tracking across multiple object classes. While immensely powerful, YOLO v9 requires significant computational resources, including high-end GPUs with large memory capacities, to process its dense multi-scale feature pyramids. Since our objective is restricted to whole-image classification rather than identifying the precise spatial coordinates of a defect within an image, employing YOLO v9 would result in unnecessary computational overhead and increased latency.

EfficientNet-B0, conversely, was designed using a neural architecture search that optimizes for both accuracy and efficiency. As established in reference literature—such as "Defective Photovoltaic Module Detection Using EfficientNet-B0 in the Machine Vision Environment" (Machines, MDPI) and "EfficientNet-b0-Based 3D Quantification Algorithm for Rectangular Defects in Pipelines" (IEEE Access)—the EfficientNet family utilizes a compound scaling method. This method uniformly scales the network's depth, width, and input resolution based on a single scaling factor. The base model, B0, utilizes Mobile Inverted Bottleneck Convolution (MBConv) blocks and Squeeze-and-Excitation (SE) optimization. This architecture allows it to maintain the high-resolution input necessary to detect minute surface anomalies (like micro-fractures and subtle scratches) while drastically reducing the number of parameters and Floating-Point Operations per Second (FLOPs) compared to heavier architectures like ResNet-50 or YOLO.

2.2 Architectural Parameters

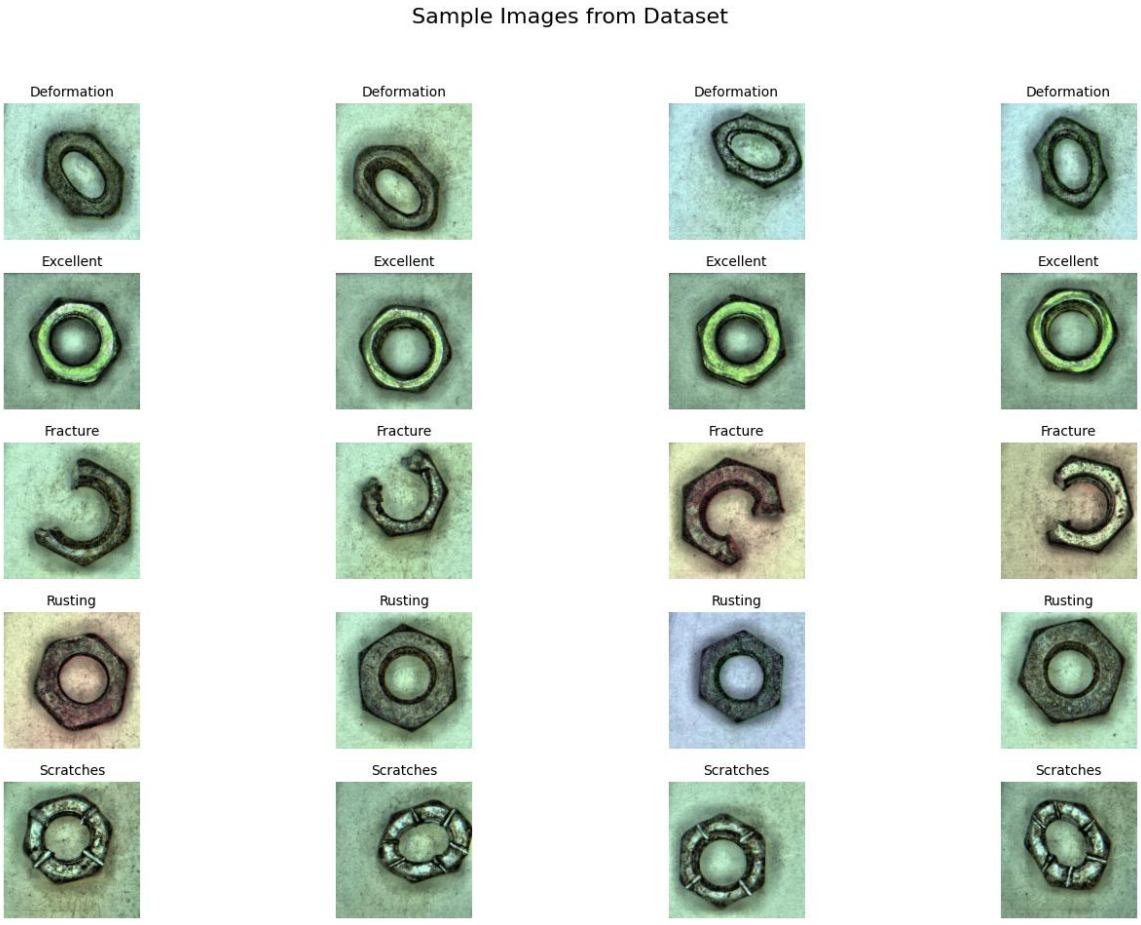
We employed the pre-trained EfficientNet-B0 model, initialized with weights trained on the ImageNet dataset. Transfer learning is crucial here, as the foundational layers already possess the ability to recognize rudimentary shapes, edges, and textures.

The classification head of the model was customized. The original 1000-class output layer was replaced with a Sequential block containing:

- A Dropout Layer with a probability (p) of 0.2. This regularization technique randomly zeroes out elements during training, forcing the network to learn robust, distributed representations and preventing overfitting.
- A fully connected Linear layer mapping the 1280 input features from the global average pooling layer to exactly 5 output features, corresponding to our defined classes.

3. Dataset Division and Data Preprocessing

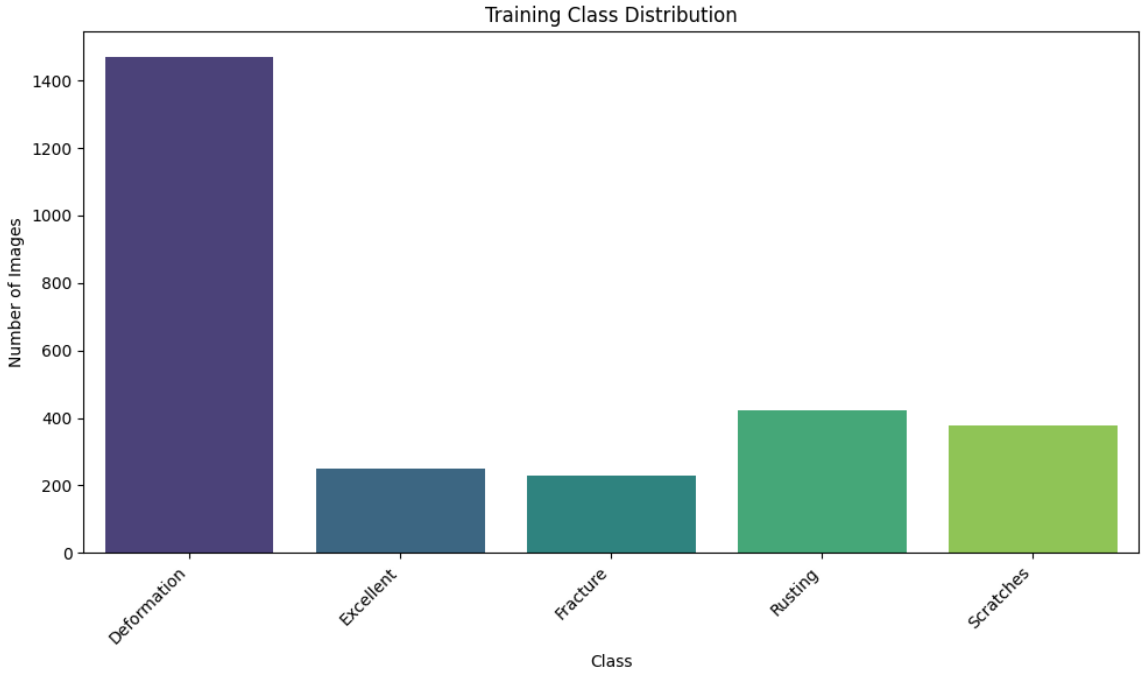
The dataset is structured into five distinct folders representing the five fine-grained classes. To evaluate the model effectively, the data is split into a Training set and a Validation set.



Class	Training Count	Validation Count	Broad Category

Deformation	1471	368	Defective
Excellent	249	63	Non-Defective
Fracture	229	58	Defective
Rusting	423	106	Defective
Scratches	378	95	Defective

The training dataset comprises a total of 2750 images, whereas the validation dataset comprises 690 images. As evident from the table, the dataset exhibits a severe class imbalance; the 'Deformation' class alone accounts for over 53% of the training data. If left unaddressed, the neural network would naturally become biased toward predicting 'Deformation'. To mitigate this, we computed normalized class weights inversely proportional to the class frequencies. These weights were integrated into the `CrossEntropyLoss` function, forcing the optimizer to penalize misclassifications of minority classes (like 'Fracture' and 'Excellent') much more heavily than majority classes.



3.2 Detailed Preprocessing Pipeline

To prepare the images for the neural network and to artificially expand the diversity of the training set (preventing overfitting), an extensive data augmentation and preprocessing pipeline was implemented using `torchvision.transforms`:

1. Random Resized Crop (224x224): The images are randomly cropped and resized to 224x224 pixels. This forces the model to recognize defects even if they are not perfectly centered.

2. Random Horizontal and Vertical Flips: Simulates objects placed on the conveyor belt in varying orientations.

3. Random Rotation (15 degrees): Accommodates slight misalignments of the nuts in the imaging environment.

4. Color Jitter: Randomly alters the brightness (0.2), contrast (0.2), saturation (0.2), and hue (0.1). This ensures the model does not rely on specific lighting conditions.

5. Tensor Conversion and Normalization: Images are converted to PyTorch tensors and normalized utilizing the standard ImageNet mean ([0.485, 0.456, 0.406]) and standard deviation ([0.229, 0.224, 0.225]).

3.3 Batch Division

The processed images are loaded into the model using a `DataLoader` with a defined Batch Size of 32. This batch size was chosen to balance memory constraints with stable gradient estimations.

- Training DataLoader: 86 batches (2750 samples / 32)
- Validation DataLoader: 22 batches (690 samples / 32)

4. Comprehensive Training Pipeline

The training strategy was separated into two specific stages. This phased approach is a best practice in transfer learning.

4.1 Stage 1: Feature Extractor Freezing (Classifier Training)

In the first stage, the parameters of the pre-trained EfficientNet-B0 feature extractor were frozen (setting `requires_grad = False`). Only the weights of the newly appended custom classifier head were updated. If we had unfrozen the entire network immediately, the large error gradients produced by the randomly initialized classifier would have backpropagated and destroyed the valuable, delicate feature representations learned from ImageNet.

- Optimizer: AdamW, a variant of the Adam optimizer that decouples weight decay, providing better generalization.
- Learning Rate: 1e-3 (A relatively high learning rate to quickly adapt the new classifier weights).
- Epochs: 10 epochs.

4.2 Stage 2: Fine-Tuning the Full Network

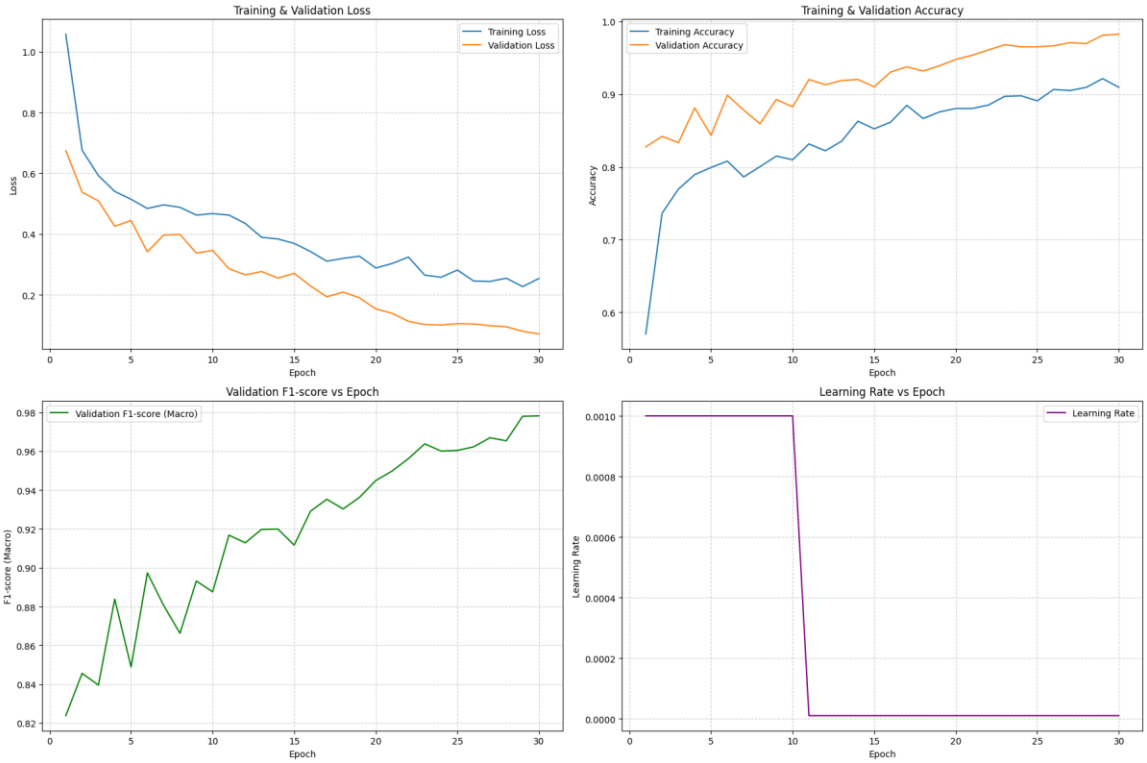
After the classifier head achieved a baseline level of competency, the entire network was unfrozen. In this stage, the optimizer subtly adjusts the pre-trained weights so that the feature extractor becomes specifically attuned to recognizing the unique textures and structures of nut defects (like rust patterns or scratch lines) rather than general ImageNet objects (like dogs or cars).

- Optimizer: AdamW

- Learning Rate: 1e-5 (A drastically reduced learning rate to ensure that the pre-trained weights are only slightly nudged, avoiding catastrophic forgetting).
- Epochs: 20 epochs.

4.3 Epoch-wise Progression

The model's learning trajectory is meticulously tracked. The following table showcases the progression across both stages.



Stage	Epoch	Train Loss	Train Acc	Train F1	Val Loss	Val Acc	Val F1
1	1	1.0579	0.5702	0.5673	0.6745	0.8275	0.8238
1	2	0.6761	0.7364	0.7199	0.5374	0.8420	0.8456
1	3	0.5921	0.7695	0.7505	0.5089	0.8333	0.8395
1	4	0.5401	0.7895	0.7719	0.4253	0.8812	0.8838
1	5	0.5143	0.7993	0.7765	0.4445	0.8435	0.8489
1	6	0.4840	0.8080	0.7869	0.3412	0.8986	0.8973
1	7	0.4959	0.7862	0.7656	0.3965	0.8783	0.8805
1	8	0.4878	0.8004	0.7775	0.3988	0.8594	0.8662
1	9	0.4622	0.8149	0.7934	0.3370	0.8928	0.8932
1	10	0.4672	0.8098	0.7912	0.3461	0.8826	0.8876
2	1	0.4623	0.8316	0.8071	0.2855	0.9203	0.9168
2	2	0.4347	0.8222	0.7998	0.2655	0.9130	0.9128
2	3	0.3892	0.8353	0.8174	0.2764	0.9188	0.9197
2	4	0.3835	0.8629	0.8416	0.2551	0.9203	0.9199

2	5	0.3690	0.8524	0.8337	0.2704	0.9101	0.9116
2	6	0.3422	0.8615	0.8427	0.2289	0.9304	0.9291
2	7	0.3102	0.8847	0.8647	0.1931	0.9377	0.9353
2	8	0.3198	0.8665	0.8493	0.2088	0.9319	0.9303
2	9	0.3269	0.8756	0.8566	0.1897	0.9391	0.9362
2	10	0.2881	0.8804	0.8612	0.1534	0.9478	0.9449
2	11	0.3027	0.8804	0.8622	0.1388	0.9536	0.9498
2	12	0.3240	0.8851	0.8610	0.1126	0.9609	0.9562
2	13	0.2645	0.8971	0.8783	0.1018	0.9681	0.9638
2	14	0.2576	0.8978	0.8794	0.1004	0.9652	0.9601
2	15	0.2813	0.8909	0.8695	0.1047	0.9652	0.9604
2	16	0.2452	0.9065	0.8900	0.1036	0.9667	0.9622
2	17	0.2439	0.9051	0.8867	0.0982	0.9710	0.9669
2	18	0.2544	0.9095	0.8917	0.0945	0.9696	0.9653
2	19	0.2267	0.9215	0.9029	0.0797	0.9812	0.9780
2	20	0.2536	0.9095	0.8893	0.0708	0.9826	0.9782

5. Detailed Evaluation Metrics

Evaluating an industrial classification model requires more than just standard accuracy, especially with imbalanced datasets.

5.1 Calculation Metrics Explained

- Precision: The ratio of correctly predicted positive observations to the total predicted positive observations. High precision relates to a low false-positive rate.
- Recall: The ratio of correctly predicted positive observations to all observations in actual class. High recall relates to a low false-negative rate.
- F1-Score: The weighted average of Precision and Recall. It is the most critical metric here, as it takes both false positives and false negatives into account.
- Support: The absolute number of samples of the true response that lie in that class.

5.2 Final Validation Statistics

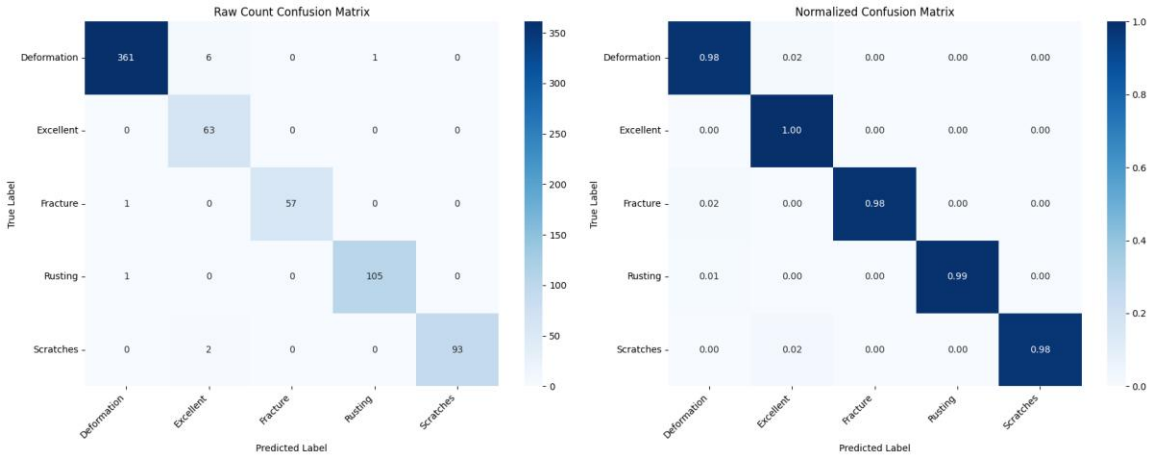
The overall Validation Accuracy reached an impressive 98.41%. The table below details the class-wise breakdown of the final model (Best Epoch: 30).

Class	Precision	Recall	F1-Score	Support
Deformation	0.9945	0.9810	0.9877	368
Excellent	0.8873	1.0000	0.9403	63
Fracture	1.0000	0.9828	0.9913	58
Rusting	0.9906	0.9906	0.9906	106
Scratches	1.0000	0.9789	0.9894	95

5.3 Deep Dive into the Confusion Matrix

The confusion matrix is a fundamental visualization tool that plots the actual labels (rows) against the predicted labels (columns).

- Why is it there and what does it represent? It provides a granular view of misclassifications. Rather than just knowing the model is 98% accurate, the confusion matrix tells us exactly "what" it is getting wrong.
- How do we know it is good? A high-performing model will have a confusion matrix characterized by very high numbers along the main diagonal (representing True Positives) and zeroes or near-zeroes in all off-diagonal cells (representing misclassifications).
- Analysis: Our model showed minor confusion, the most prominent being 'Scratches' incorrectly predicted as 'Excellent' in 2.11% of the scratch cases. Understanding these specific failure points allows engineers to fine-tune lighting or camera angles specifically to highlight scratch depth in future iterations.



6. Real-World Model Predictions and Inference

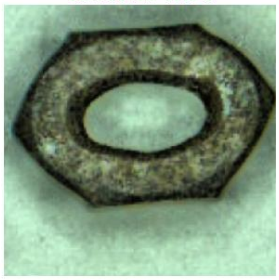
6.1 Analyzing Predictions

By visually inspecting the model's output on the validation set, we divide the results into two categories:

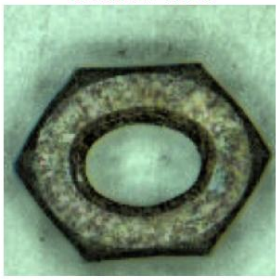
- Correct Predictions: In these cases, the model accurately assigns the defect class with remarkably high confidence (e.g., classifying a heavily rusted nut as 'Rusting' with 99.63% confidence).
- Incorrect Predictions: Analyzing these is crucial for debugging. For instance, a very faint surface scratch might be visually indistinguishable from a normal manufacturing artifact, leading the model to confidently but incorrectly label it as 'Excellent'.

Predicted: Deformation
Actual: Deformation
Confidence: 99.7%

Predicted: Deformation
Actual: Deformation
Confidence: 99.3%



Predicted: Deformation
Actual: Deformation
Confidence: 97.5%



Predicted: Deformation
Actual: Deformation
Confidence: 99.3%

Predicted: Deformation
Actual: Deformation
Confidence: 61.9%

Predicted: Deformation
Actual: Deformation
Confidence: 92.6%

Predicted: Deformation
Actual: Deformation
Confidence: 99.3%

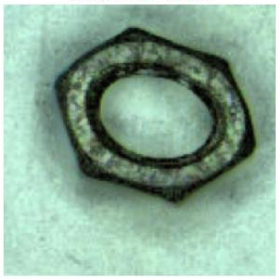
Predicted: Deformation
Actual: Deformation
Confidence: 96.9%

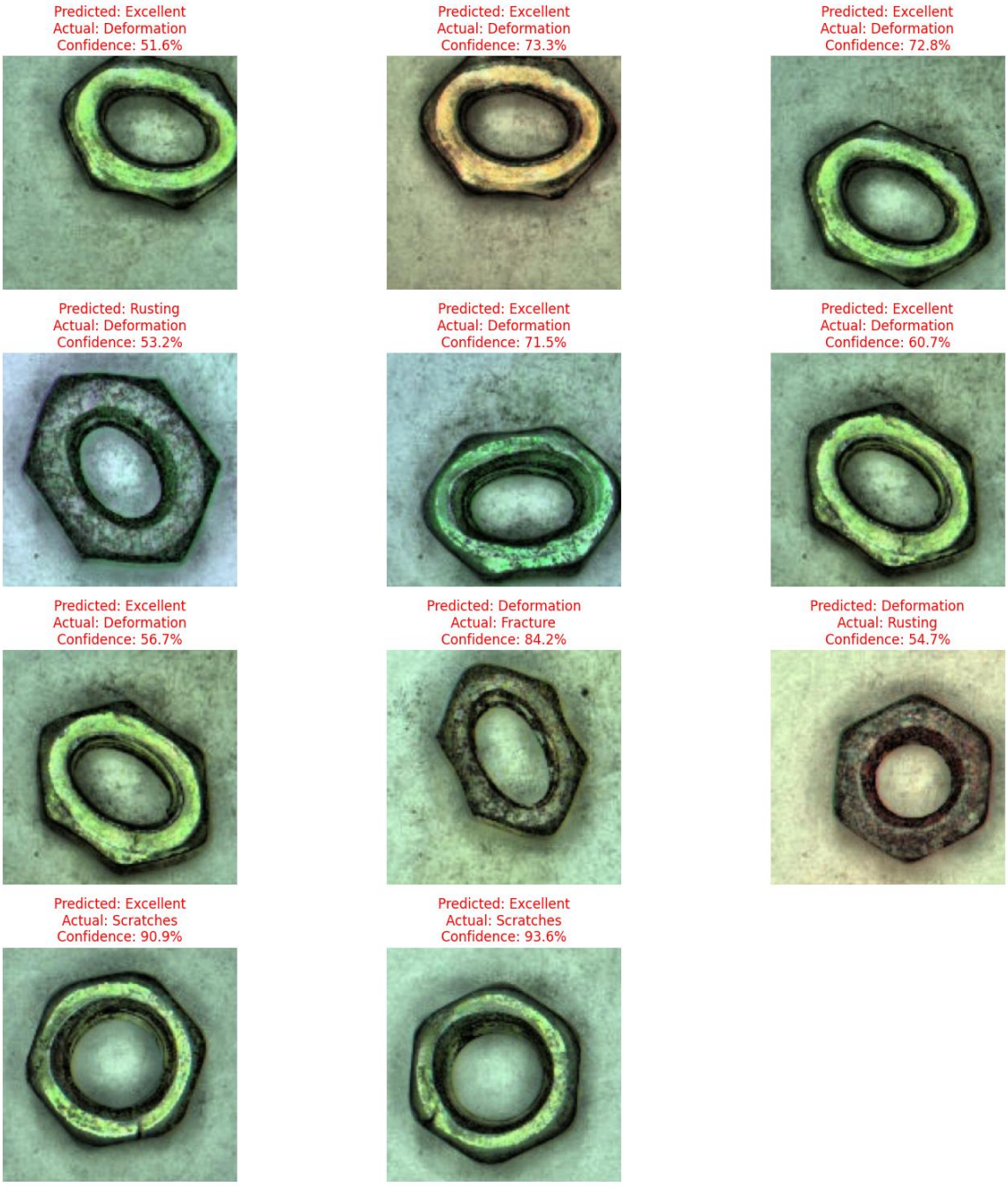
Predicted: Deformation
Actual: Deformation
Confidence: 99.1%

Predicted: Deformation
Actual: Deformation
Confidence: 97.4%

Predicted: Deformation
Actual: Deformation
Confidence: 99.1%

Predicted: Deformation
Actual: Deformation
Confidence: 76.5%





6.2 The Single-Line Prediction Function

To operationalize this trained neural network on a factory floor, a dedicated `predict_image` function is established.

- Purpose: This function abstracts away all the complexity. It accepts a single raw image file path, automatically applies the rigid validation preprocessing transformations, converts it to a tensor, and passes it through the model in inference mode (`torch.no_grad()`).

- Output: It processes the raw logit outputs through a Softmax function to generate easily interpretable confidence percentages (e.g., "Deformation: 90.62%"). This single-line function is the literal bridge between theoretical data science and practical, automated software engineering in the manufacturing pipeline.

7. References

1. Nut Surface Defect Dataset, sourced from Kaggle:
https://www.kaggle.com/datasets/weihaoreal/nut-surface-defect-dataset

2. Wu, D., Hong, Y., Wang, J., Wu, S., Zhang, Z., & Liu, Y. (2025). EfficientNet-b0-Based 3D Quantification Algorithm for Rectangular Defects in Pipelines. IEEE Access.

3. Shin, M., Seo, J., Lee, I.-B., & Kim, S. (2026). Defective Photovoltaic Module Detection Using EfficientNet-B0 in the Machine Vision Environment. Machines, MDPI.